

Lesson 4 — Pseudocode, Abstraction & Iterative Design

Matches LEGO Teacher Portal lesson: "Supplies from the Skies".

Objective (I can...): I can write pseudocode, use abstraction to hide detail, and improve my program across two test-and-revise cycles. **CS concept:** Pseudocode and abstraction; the iterative design loop (plan → build → test → refine) · **Builds on:** Lesson 3 (decomposition and flowcharts).

AI Connection (2 min): Engineers spec an AI feature in plain language first — "IF a face is recognized with high confidence, THEN unlock" — then implement and refine it over many iterations. *First-principles question:* "Why is the first version of almost anything engineered never the last?"

Materials: LEGO CS & AI kit + device per pair, plan sheets, notebooks. **Model reuse:** Basics model from Lesson 2 — code plus one small tweak.

60-minute flow - Warm-up (8): "Spec swap" — each pair writes 4–5 lines of pseudocode for a behavior, then trades with another pair; your received spec is your mission. Ambiguous specs get built wrong on purpose — that's the lesson in precision. - **Concept + demo (10):** Coach writes pseudocode, maps it to blocks, runs it, finds it imperfect, and revises — narrating the iterative loop. Name *abstraction*: the plan hides motor ports and timings so you can reason about the big steps. (*Rotate format:* "coach ships a bug, team catches it.") - **Build/Code (30):** Pairs build the received spec: refine pseudocode, code it, run it, then run **at least one deliberate revision** and log in the notebook what changed and why. **Make-it-yours beat:** adapt one brick to the mission theme. *Swap at 15 min.* - **Challenge (7):** Change one line of the spec (e.g., add a condition in words) and update the code to match. - **Share + pack-up (5):** Pairs read the pseudocode aloud, run to prove the match, and state their one revision. Model stays built.

Pair roles: Navigator owns the plan sheet and the revision log; Driver translates and runs. Swap at 15 min. (*Trio: third logs the revision rationale.*)

Differentiation: *Support* — sentence-framed pseudocode to complete. *Extension / fast-finisher* — write pseudocode that includes a repeated step and a decision in words. *Reteach* — code one line together, then let them continue; do the revision together.

Exit check: Pair's pseudocode maps one-to-one to working blocks, and they can name one revision and the reason for it. **At-home (optional):** Write pseudocode for a daily task, then write a v2 that fixes one weakness. ~10 min.